COURSEWORK ASSIGNMENT SUBMISSION

Name: Daksh Srivastava **Roll No:** 150096725087
Subject: Software Engineering & Project Management **Day:** Day 16 — Software Architecture & Architectural Design

---

Section 9: Classroom Activity — Design the CEMS Architecture

System architecture specification for the College Event Management System (CEMS).

1. Architectural Pattern Selection

- **Selected Architectural Style:** 3-Tier Layered Architecture with Asynchronous Message Worker Decoupling.
- **Engineering Justification:** Provides strict separation between user presentation, transactional business domain rules, and data persistence. A 3-tier model is easy to deploy, simple to debug, and supports independent vertical scaling of backend API instances behind a load balancer during peak campus registration spikes.

2. Architectural Component Hierarchy

Architectural Tier	Physical Component	Technology Stack	Core Responsibilities
Presentation Tier	Client Web App (SPA)	React.js / TailwindCSS	Render UI, validate input formats, display QR tickets, communicate via REST/JSON.
Application / API Tier	Application Server	Node.js (Express) / Go	Execute business logic, enforce RBAC, orchestrate database transactions, manage queues.
Integration Tier	Async Message Broker	Redis BullMQ / RabbitMQ	Buffer email/SMS dispatch jobs and decouple third-party network latency from HTTP responses.
Data Tier	Relational Database	PostgreSQL 16	Store relational tables (Users, Events, Registrations, Logs) with ACID transaction guarantees.

3. Event Registration Data Flow

1. Client Browser submits `POST /api/v1/registrations` with student JWT and `eventId`. 2. API Gateway validates JWT signature and passes payload to `BookingController`. 3. `BookingService` opens a PostgreSQL database transaction with `SELECT … FOR UPDATE` on the event row. 4. If current seats <

capacity, an entry is inserted into `registrations`, seat count increments, and transaction commits. 5. An asynchronous job is dispatched to Redis queue for email generation. 6. API returns HTTP 201 Created with confirmation payload to Client Browser.

Section 12: Homework — Architecture Analysis & ADR

Formal architectural comparison and Architecture Decision Record (ADR) for enterprise campus systems.

1. Architectural Decision Record: ADR-001 (Relational vs Document Store)

- **Context:** CEMS requires strict seat capacity enforcement where two concurrent requests for the final available seat must never result in overbooking.
- **Decision:** Select PostgreSQL (Relational Database) over MongoDB (NoSQL Document Store).
- **Rationale:** PostgreSQL guarantees ACID transactions with strict serializable isolation levels and row-level locks. MongoDB document-level atomicity requires complex two-phase commit patterns across collections to guarantee that attendee records and event seat counters remain synchronized under extreme concurrency.
- **Consequences:** Requires strict upfront schema definitions and database migration management, but guarantees zero seat over-allocation.

2. Comparative Architectural Analysis: Layered Architecture vs MVC

- **Layered Architecture:** Organizes software horizontally by technical responsibility (Presentation -> Business Logic -> Data Access -> Database). Concerns the entire system topology.
- **Model-View-Controller (MVC):** Organizes the user-interface layer by separating data representations (Model), visual displays (View), and input coordination (Controller).
- **Coexistence:** Layered architecture and MVC are complementary rather than mutually exclusive. MVC frequently serves as the internal design pattern inside the Presentation and Application layers of a broader 3-Tier Layered Architecture.